

AutoRef — Technical Documentation

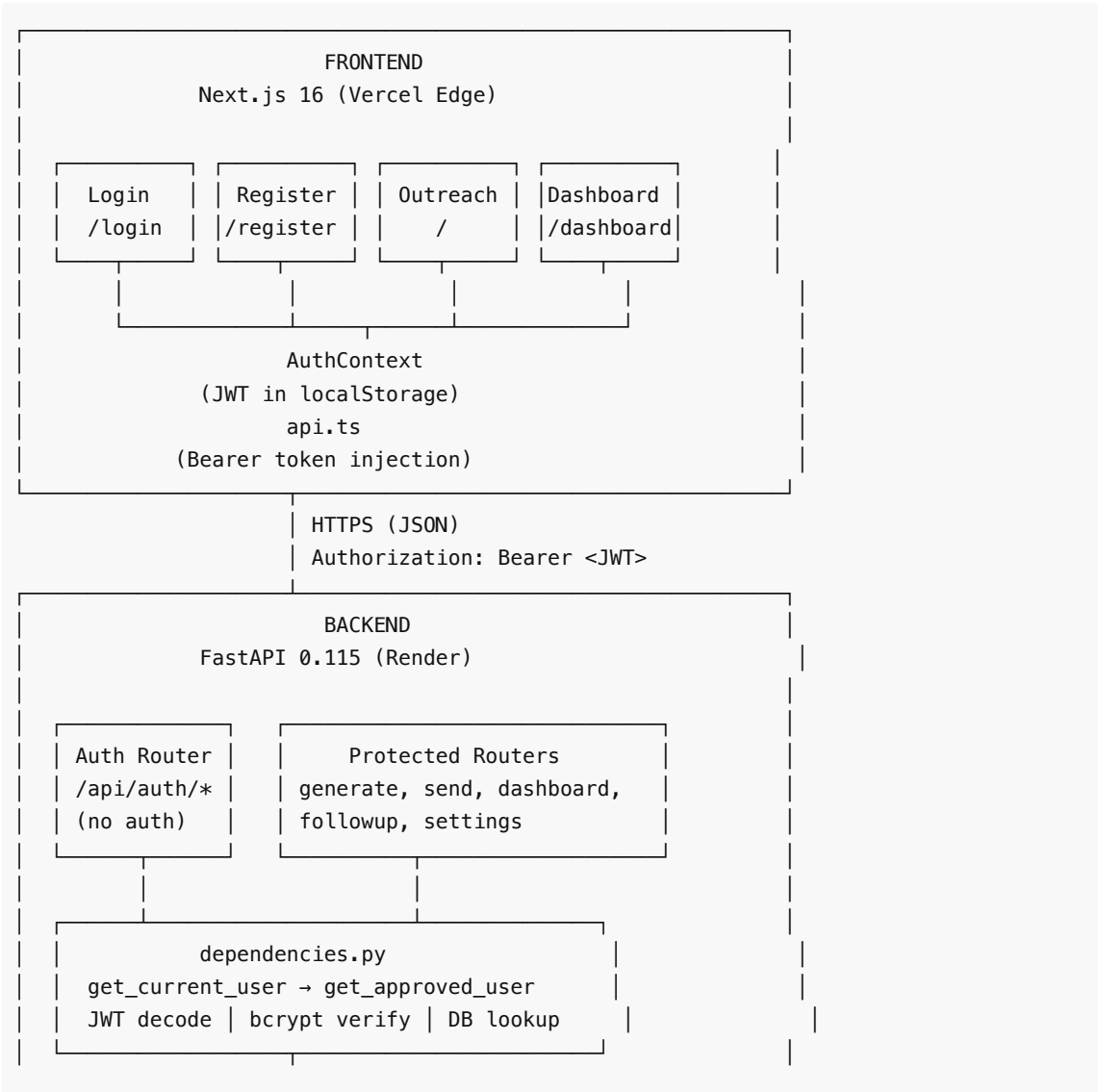
1. Project Overview

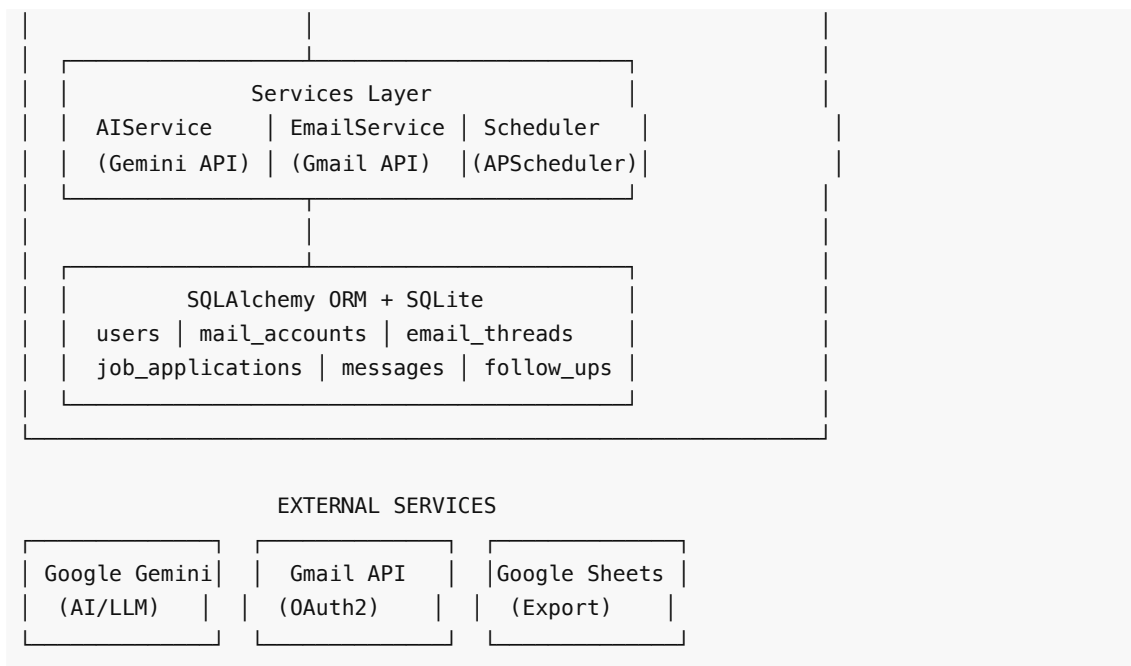
AutoRef is a full-stack SaaS platform that automates cold outreach for job seekers. It uses LLM-powered email generation (Google Gemini) to semantically match resumes against job descriptions, generates personalized referral emails, sends them via Gmail OAuth, and orchestrates automated follow-up sequences — all from a single dashboard.

Key Metrics:

- Reduces cold email drafting from ~15 minutes to ~45 seconds (95% reduction)
- Supports 3 target role categories with dedicated resume matching
- Automated 3-stage follow-up pipeline with reply detection
- Multi-tenant architecture with admin approval flow

2. System Architecture





3. Authentication & Authorization

3.1 Architecture — Three-Tier Dependency Chain



Every protected endpoint uses `get_approved_user` as a FastAPI `Depends()`. Admin-only endpoints (approve/reject users) use `get_admin_user`.

3.2 JWT Token Flow

Token	Lifetime	Purpose
-------	----------	---------

Access Token	24 hours	API authentication via Authorization: Bearer <token>
Refresh Token	7 days	Obtain new access token without re-login

Token payload:

```
{
  "sub": "42",          // user_id as string
  "exp": 1716000000,    // expiry timestamp
  "type": "access"      // or "refresh"
}
```

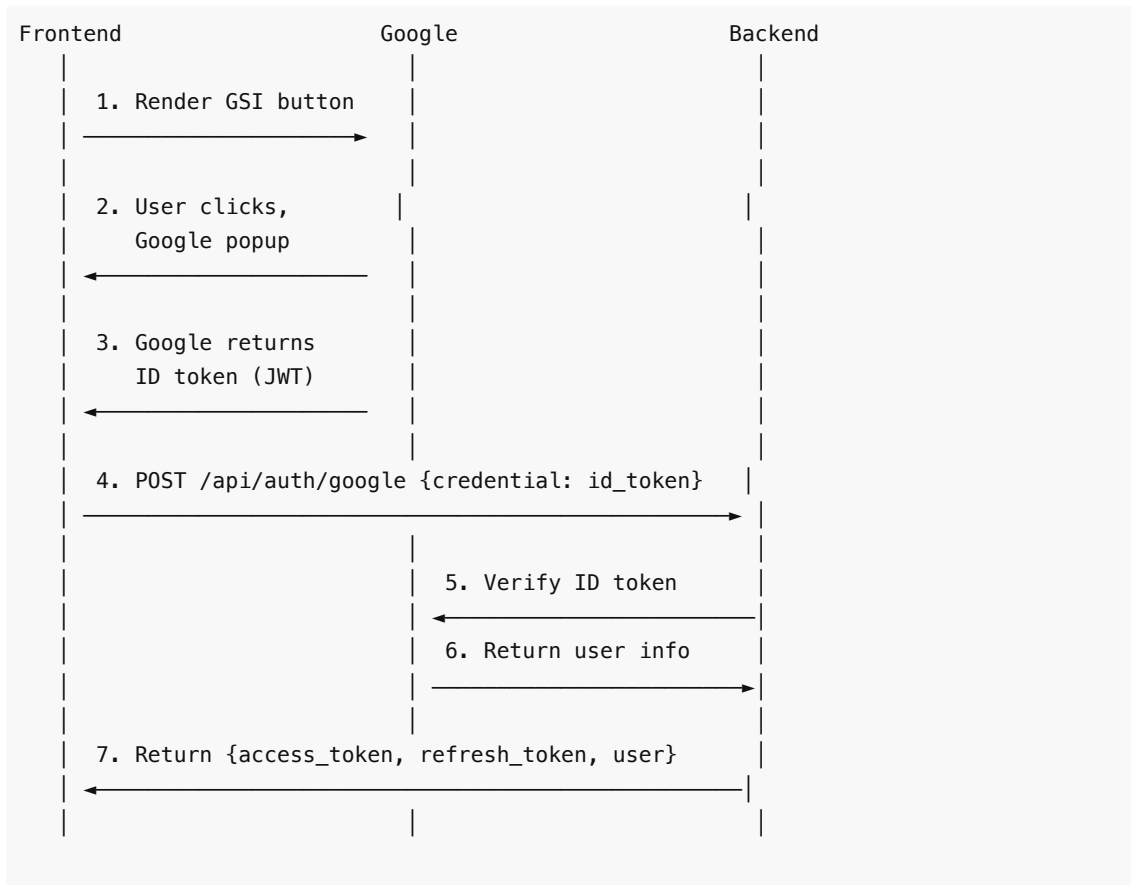
Why HS256 (symmetric) over RS256 (asymmetric)?

- Single backend server — no need for public key distribution
- Simpler config (one secret vs key pair)
- Lower computational overhead per request

3.3 Password Security

- **Algorithm:** bcrypt via `passlib`
- **Why bcrypt over argon2?** Wider library support, battle-tested, sufficient for this scale
- Passwords are **never stored in plaintext** — only the bcrypt hash
- `password_hash` is nullable to support Google OAuth-only accounts

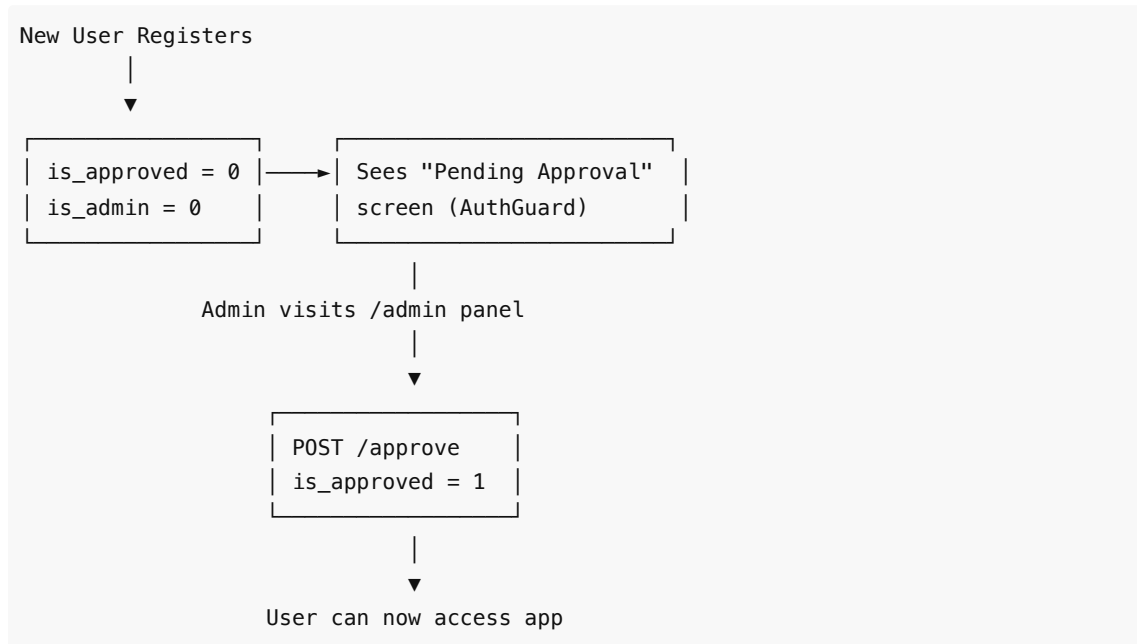
3.4 Google Sign-In Flow



- | 8. Store in localStorage |
- | 9. Redirect to / |

Key: The Google ID token is verified server-side using `google.oauth2.id_token.verify_oauth2_token()`. We never trust the client.

3.5 Admin Approval Flow



First-user bootstrap: The very first registered user OR the user matching `ADMIN_EMAIL` env var is auto-approved as admin.

3.6 Gmail OAuth (Mail Connection) — State Parameter Fix

Problem: Gmail OAuth callback is a redirect from Google — no Bearer token attached.

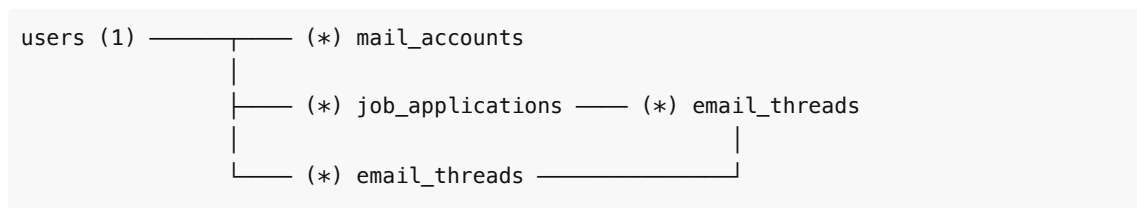
Solution: We pass `user_id` in the OAuth `state` parameter:

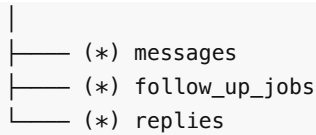
1. GET `/api/auth/gmail` → passes `state=current_user.id` to Google
2. Google redirects back to `/api/auth/gmail/callback?code=xxx&state=42`
3. Callback reads `state` → looks up user by ID → links Gmail account

Scope relaxation: We set `OAUTHLIB_RELAX_TOKEN_SCOPE=1` because Google sometimes returns scope strings in a different order than requested, causing the library to throw "Scope has changed" errors.

4. Multi-Tenant Data Isolation

Every data-bearing table includes `user_id` FK:





Every query is scoped:

```

# Dashboard – only shows current user's threads
db.query(EmailThread).filter(EmailThread.user_id == current_user.id)

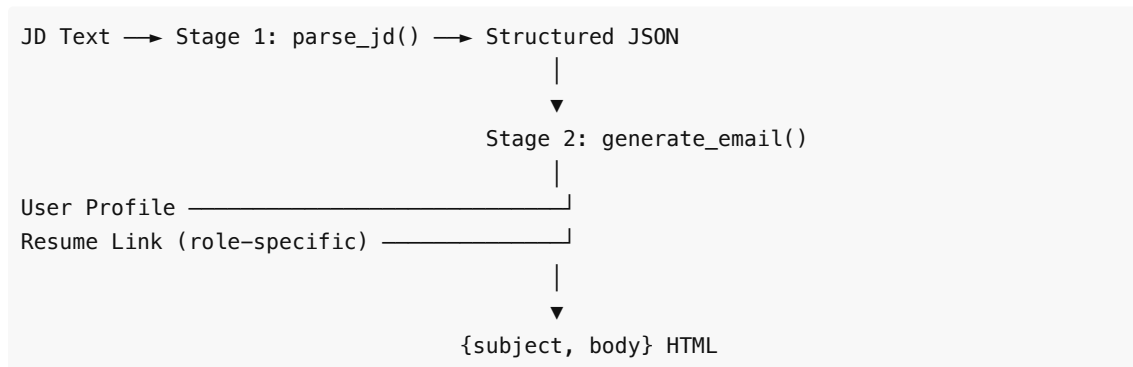
# Status update – verifies ownership before mutation
thread = db.query(EmailThread).filter(
    EmailThread.id == thread_id,
    EmailThread.user_id == current_user.id,
).first()
  
```

Trade-off: Row-level filtering vs separate databases per user

- Row-level filtering (our approach): simpler, works with SQLite, sufficient for current scale
- Separate DBs: better isolation but complex operationally; overkill for <1000 users

5. AI Email Generation Pipeline

5.1 Two-Stage Pipeline



5.2 Role-Specific Prompt Engineering

Role	Bullet Categories	Subject Style	Resume Link
Backend/SDE	API Design, Performance, DSA	"800+ TPS API at p99 <50ms"	SDE resume
Fintech	Payment Systems, Security, Reliability	"ACID Transactions & Idempotent APIs"	Fintech resume
Data Engineering	Pipeline Architecture, Spark, Data Modeling	"150 GB/day ingestion pipeline"	DE resume

Each role gets a tailored `role_configs` dict with:

- `bullet_guidance` : Specific achievement categories to extract from the user's profile
- `subject_examples` : Domain-appropriate subject line templates
- `emphasis` : Directive telling the LLM which angle to frame the profile through

5.3 Retry & Rate Limiting

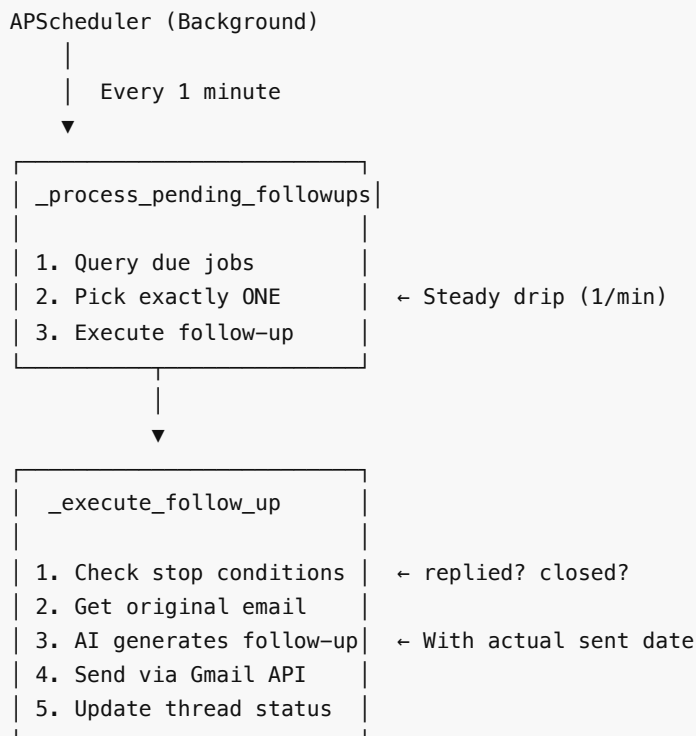
```
def _call_gemini(self, prompt, max_retries=3):
    for attempt in range(max_retries):
        try:
            return client.models.generate_content(...)
        except Exception:
            if attempt < max_retries - 1:
                time.sleep(2 ** attempt) # Exponential backoff
            else:
                raise
```

Trade-off: Synchronous calls with retry vs async queue

- Current: Synchronous with exponential backoff — simpler, user sees result immediately
- Alternative: Celery/RQ queue — better for high concurrency, adds infrastructure complexity

6. Follow-Up Automation Engine

6.1 Architecture



6.2 Key Design Decisions

Why process only 1 per minute?

- Gmail rate limits: sending too many emails quickly triggers spam flags
- Natural throttling without complex threading/semaphore logic
- 1/min = 60/hour = more than enough for individual job seekers

Why APScheduler over Celery?

- No external broker needed (Redis/RabbitMQ)
- Runs in-process — zero infrastructure overhead
- Sufficient for single-server deployment on Render free tier

Idempotency guarantees:

- Each `FollowUpJob` has a unique `(thread_id, follow_up_number)` pair
- Jobs transition: `pending` → `sent` | `failed` | `cancelled` (terminal states)
- Failed jobs are NOT retried (prevents spam loops from permanent failures)

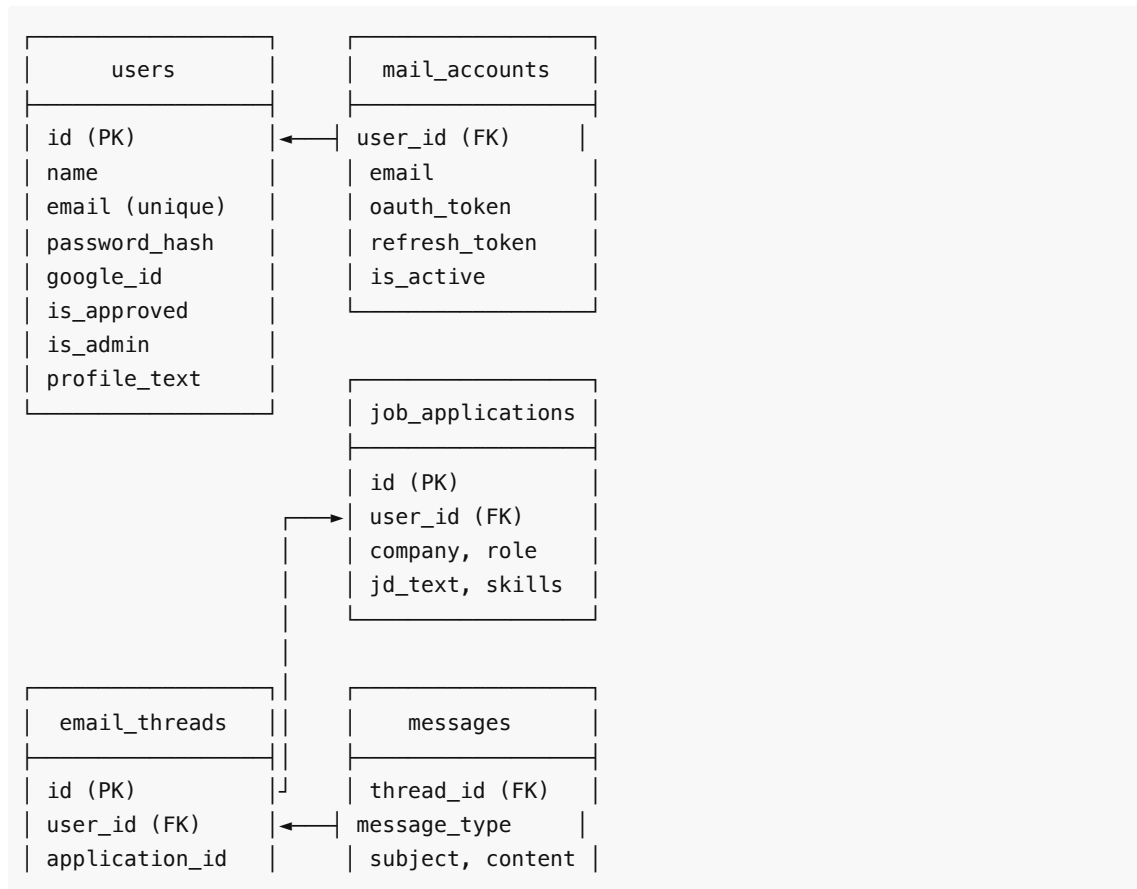
6.3 Placeholder Hallucination Prevention

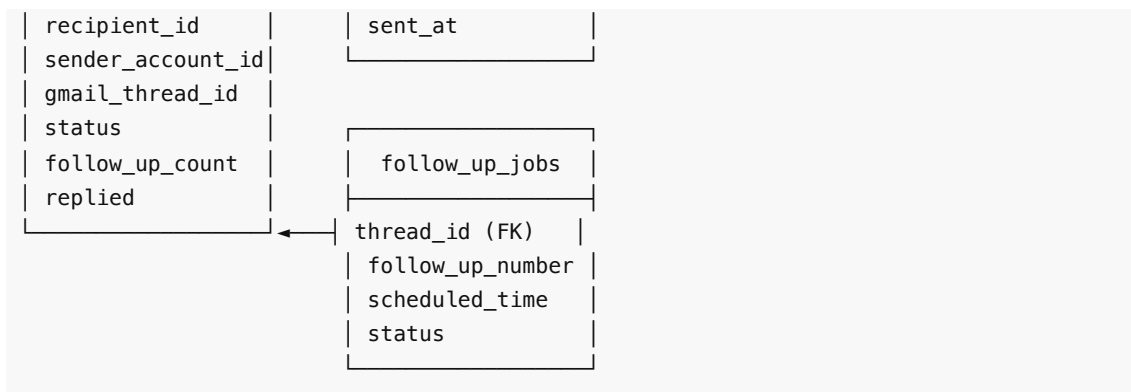
Follow-up emails had LLM-generated `[Date of original email]` placeholders.

Three-layer defense:

1. **Data injection:** Pass `original_sent_date` (e.g., "May 14, 2026") into the prompt
2. **Prompt engineering:** Rule 8 instructs the LLM to use vague phrasing ("my recent email")
3. **Catch-all regex:** `re.sub(r'\[.*?\]', '', result)` strips ANY bracket-wrapped placeholder

7. Database Schema





Why SQLite over PostgreSQL?

- Zero operational cost (no managed DB service)
- File-based — works on Render free tier without add-ons
- Sufficient for single-user/small-team usage (<10K rows)
- Trade-off: No concurrent write scaling — acceptable for this workload

8. API Endpoints

Auth (No auth required)

Method	Endpoint	Purpose
POST	/api/auth/register	Email/password registration
POST	/api/auth/login	Email/password login
POST	/api/auth/google	Google ID token login
POST	/api/auth/refresh	Refresh access token
GET	/api/auth/me	Get current user profile

Protected (Requires `get_approved_user`)

Method	Endpoint	Purpose
POST	/api/generate-email	Parse JD + generate email
POST	/api/send-email	Send email via Gmail
GET	/api/dashboard	List user's outreach records
GET	/api/dashboard/stats	Aggregate statistics
POST	/api/update-status	Update thread status
DELETE	/api/thread/{id}	Delete a thread
POST	/api/schedule-followups	Schedule follow-ups
POST	/api/stop-followups	Cancel pending follow-ups

GET	/api/profile	Get user profile
POST	/api/profile	Update user profile
GET	/api/mail-accounts	List Gmail accounts
GET	/api/auth/gmail	Initiate Gmail OAuth

Admin (Requires `get_admin_user`)

Method	Endpoint	Purpose
GET	/api/auth/pending-users	List unapproved users
POST	/api/auth/approve-user	Approve/reject a user

9. Frontend Architecture

Route Protection

```

layout.tsx
├─ AuthProvider (Context)
│   │
│   ├─ /login      ← Public (redirects to / if logged in)
│   ├─ /register   ← Public (redirects to / if logged in)
│   │
│   ├─ /          ← AuthGuard → Outreach page
│   ├─ /dashboard ← AuthGuard → Dashboard
│   ├─ /settings  ← AuthGuard → Settings
│   └─ /admin     ← AuthGuard + is_admin check → Admin panel

```

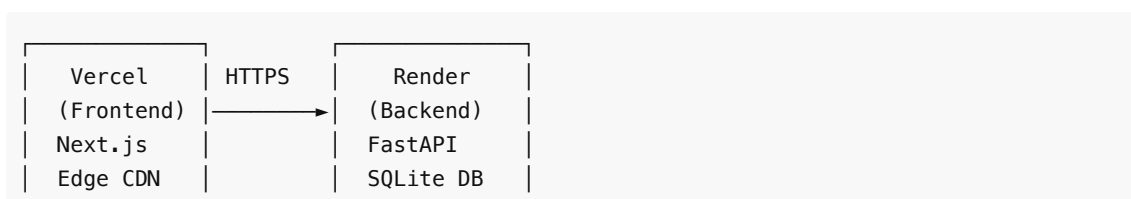
AuthGuard States

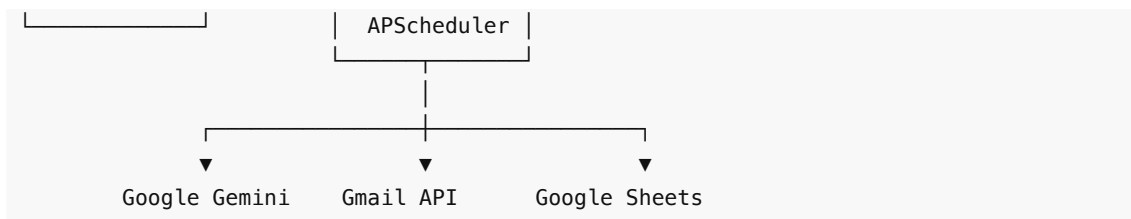
1. **Loading** → Spinner (validating token via `/api/auth/me`)
2. **Not authenticated** → Redirect to `/login`
3. **Authenticated but not approved** → "Pending Approval" screen
4. **Authenticated and approved** → Render children

Token Management

- Stored in `localStorage` : `autoref_token` , `autoref_refresh_token` , `autoref_user`
- Injected via `api.ts` on every request: `Authorization: Bearer <token>`
- **401 interceptor**: Auto-clears auth and redirects to `/login`

10. Deployment Architecture





Keep-alive ping: Render free tier spins down after 15 min inactivity. APScheduler pings `/health` every 14 minutes to prevent this.

11. Design Trade-offs & Interview Defense

Q: "Why SQLite instead of PostgreSQL?"

A: Operational simplicity. SQLite is zero-config, file-based, and free. For a single-tenant SaaS with <10K rows and no concurrent write pressure (one user at a time), SQLite outperforms managed Postgres on cost (\$0 vs \$7+/month) and latency (no network hop). Migration path to Postgres is trivial — SQLAlchemy abstracts the dialect.

Q: "Why not use Celery for background jobs?"

A: APScheduler runs in-process with no external broker dependency. For a single-server deployment sending ~60 emails/hour max, the overhead of Redis + Celery worker processes is unjustified. APScheduler's `BackgroundScheduler` gives us cron-like scheduling with 5 lines of code.

Q: "Why JWT over session cookies?"

A: Decoupled frontend (Vercel) and backend (Render) — different domains. Session cookies require `SameSite=None` + complex CORS. JWTs in `Authorization` headers work cleanly across origins. Trade-off: JWTs can't be server-side revoked without a blacklist; we accept this since tokens expire in 24h.

Q: "Why localStorage for tokens?"

A: Simpler than httpOnly cookies for a cross-origin SPA. XSS risk is mitigated by: (1) Next.js auto-escapes JSX, (2) no `dangerouslySetInnerHTML` on user input, (3) CSP headers on Vercel. For a portfolio project, this is an acceptable trade-off over the complexity of a BFF (Backend-For-Frontend) cookie proxy.

Q: "How do you prevent abuse?"

A: Three layers:

1. **Admin approval:** New users can't access the app until manually approved
2. **Rate throttling:** Follow-ups limited to 1/minute via APScheduler's steady drip
3. **Scope isolation:** Users only see/modify their own data (row-level filtering on `user_id`)

Q: "Why prompt engineering over fine-tuning for email generation?"

A: Fine-tuning requires curated training data (hundreds of labeled email pairs) and ongoing model management. Prompt engineering with `gemini-2.5-flash-lite` gives us 90% of the quality at 0% of the training cost. The role-specific `role_configs` dict acts as a lightweight "soft fine-tune" — it injects domain vocabulary and category guidance without model modification.

Q: "What happens if the LLM hallucinates?"

A: Three-layer defense:

- 1. **Prompt rules:** "DO NOT hallucinate projects, metrics, or experiences I do not have"
- 2. **Factual grounding:** User's `profile_text` is injected as the only source of truth
- 3. **Post-processing regex:** Catch-all `\[.*?\]` strips any remaining placeholder artifacts

12. Environment Variables Reference

Backend (Render)

Variable	Purpose	Example
DATABASE_URL	SQLite/Postgres connection	sqlite:///./autoref.db
GEMINI_API_KEY	Google Gemini API key	AIza...
GOOGLE_CLIENT_ID	OAuth Client ID	123...apps.googleusercontent.com
GOOGLE_CLIENT_SECRET	OAuth Client Secret	G0CSPX-...
GOOGLE_REDIRECT_URI	Gmail OAuth callback URL	https://api.example.com/api/auth/gmail/callback
JWT_SECRET_KEY	JWT signing secret	Random 64-char string
ADMIN_EMAIL	Auto-approved admin email	you@gmail.com
FRONTEND_URL	Frontend origin for CORS	https://app.vercel.app

Frontend (Vercel)

Variable	Purpose
NEXT_PUBLIC_API_URL	Backend API base URL
NEXT_PUBLIC_GOOGLE_CLIENT_ID	Google Sign-In button